FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL

ASSESSMENT . 2

RULE – BASED AGENT

Student Name	Coritana, Rembrandt V.
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf** *(Example: TA2_Patilano.pdf)*
8. Submit the **python file** and **PDF** file.

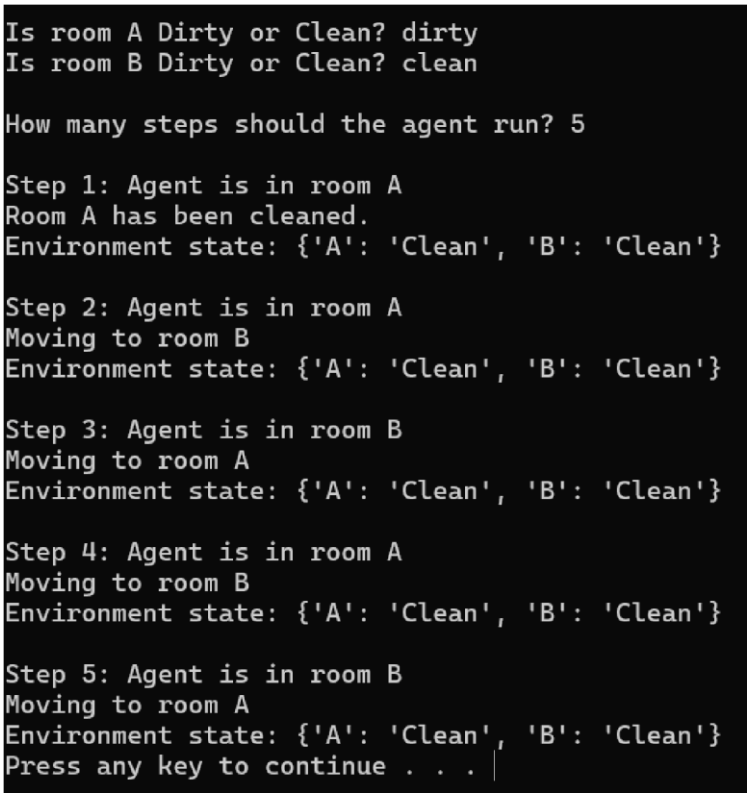
Laboratory Task Pre-Lab Preparation

1. Review the Python concepts:
    - Classes and objects
    - If-Else statements
    - Loops (for loop)
    - Functions / methods
2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Set Up the Environment

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

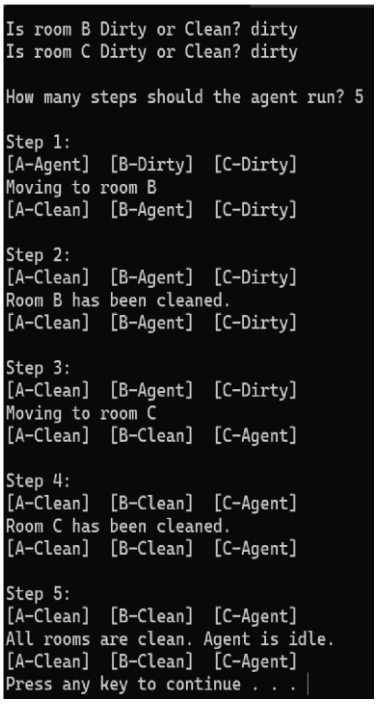
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks

```
Enter the state of Room A (dirty/clean): a
Invalid input. Please enter 'dirty' or 'clean'.
Enter the state of Room A (dirty/clean): dirty
Enter the state of Room B (dirty/clean): dirty

Initial Environment:
Room A: dirty
Room B: dirty
Enter the number of steps: 5

--- Simulation Started ---

Step 1
Agent Location: Room A
Action: Room A was cleaned.
Environment State:
  Room A: clean
  Room B: dirty

Step 2
Agent Location: Room A
Action: Moving to Room B.
Environment State:
  Room A: clean
  Room B: dirty

Step 3
Agent Location: Room B
Action: Room B was cleaned.
Environment State:
  Room A: clean
  Room B: clean

Step 4
Agent Location: Room B
Action: Moving to Room A.
Environment State:
  Room A: clean
  Room B: clean

Step 5
Agent Location: Room A
Action: Moving to Room B.
Environment State:
  Room A: clean
  Room B: clean

--- Simulation Finished ---
The vacuum agent has completed the simulation.
```

Code:

```python
import time


# Get the number of simulation steps
def ask_steps():
    while True:
        try:
            steps = int(input("Enter the number of steps: "))

            if steps > 0:
                return steps
            print("Please enter a number greater than 0.")
```

```
        except ValueError:
            print("Invalid input. Please enter a whole number.")


# Get the initial state of each room
def get_room_state(room_name):
    while True:
        state = input(f"Enter the state of {room_name} (dirty/clean):
").strip().lower()
         if state in ["dirty",
"clean"]:
            return state

        print("Invalid input. Please enter 'dirty' or 'clean'.")


class VacuumAgent:

    def __init__(self, environment):
        self.environment = environment
        self.position = 0

    def current_room(self):
        return self.environment[self.position]

    def is_dirty(self):
        return self.current_room()["state"] == "dirty"

    def clean_room(self):
        room = self.current_room()
        room["state"] = "clean"
        print(f"Action: {room['name']} was cleaned.")

    def move(self):
        # Switch between Room A and Room B
        self.position = 1 - self.position
        print(f"Action: Moving to {self.current_room()['name']}.")
```

```
    def perceive_and_act(self):
        if self.is_dirty():
            self.clean_room()
else:
            time.sleep(0.5)
            self.move()

# ----------------------------
# Create the environment
# ----------------------------

room_a = {
    "name": "Room A",
    "state": get_room_state("Room A")
}
 room_b =
{
    "name": "Room B",
    "state": get_room_state("Room B")
}

environment = [room_a, room_b]


# Display initial room conditions
print("\nInitial Environment:")
for room in environment:
    print(f"{room['name']}: {room['state']}")


# Get simulation length
total_steps = ask_steps()

agent = VacuumAgent(environment)
```

```
# ----------------------------
# Run the agent
# ----------------------------

print("\n--- Simulation Started ---")

for step in range(1, total_steps + 1):

    print(f"\nStep {step}")
    print(f"Agent Location: {agent.current_room()['name']}")

    agent.perceive_and_act()

    print("Environment State:")
    for room in environment:
        print(f"  {room['name']}: {room['state']}")


print("\n--- Simulation Finished ---")
print("The vacuum agent has completed the simulation.")
```

Task 4: Analysis Questions
After running the simulation, answer the following:

1. What rules does the agent follow?
   - The agent follows two main rules. If the current room is dirty, it cleans the room. If the room is already clean, it moves to the next room. Both rules are implemented as the two possible actions in the perceive_and_act() method.

2. How does the agent decide when to clean and when to move?

   At each step, the agent checks the condition of the room it is currently in, using the is_room_dirty() method. If the room is dirty, the agent cleans it. If the room is clean, the agent moves to the next room.

3. What happens if both rooms are already clean at the start?

   If both rooms are clean from the beginning, the agent has nothing to clean. It simply moves from one room to the other. Since it always finds the current room clean, it keeps alternating between Room A and Room B for the number of steps the user specified.

4. How would you modify the agent to handle more than two rooms?

   The agent can handle more rooms by adding more room dictionaries to the collection of rooms. The move_to_next_room() method can then use (index + 1) % len(self.rooms) to cycle through all the available rooms. For example, if a Room C is added, the agent moves from Room A to Room B, then to Room C, and back to Room A.